

Relational Databases & SQL

Part III · Data & Storage — the trusted default for structured, related data.

LEARNING OBJECTIVES

- Explain the relational model: tables, rows, columns, primary keys, foreign keys.
- Read and write basic SQL, and explain why JOINS are the relational superpower.
- Explain how a schema and constraints enforce data integrity.
- Explain, at a high level, why relational databases are trusted (ACID) — and where they're rigid.
- Decide when a relational database is the right choice.

REAL-WORLD ANALOGY — A SET OF CROSS-REFERENCED LEDGERS

Imagine a company's records kept as several neat ledgers: one lists customers (each with a customer number), another lists orders (each order noting *which* customer number placed it). You never rewrite a customer's details on every order — you just reference their number. To answer “what did customer 42 order?” you look up 42 in the orders ledger. A **relational database** is exactly this: separate tables that reference each other by ID, so data is stored once and questions can be answered by following the references.

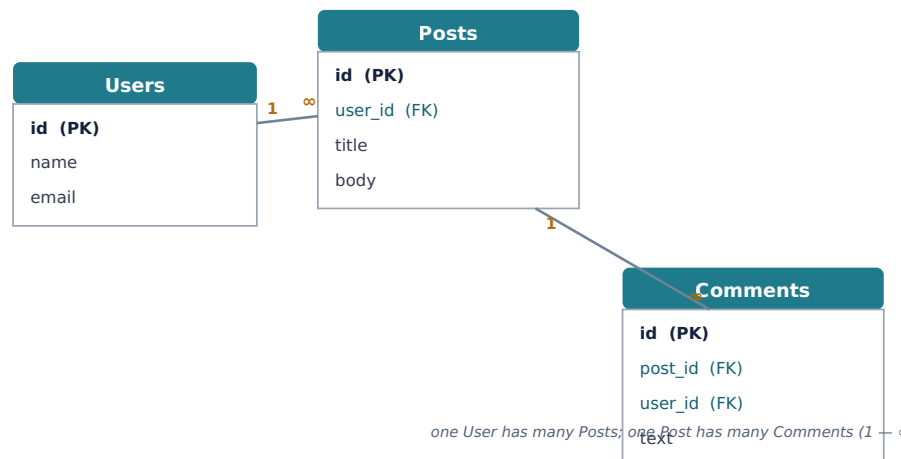
11.1 The problem relational databases solve

Part II scaled the app tier; now we confront storing data **durably and correctly**. Most real data is **structured** (it has a fixed shape) and **related** (users have posts, posts have comments, orders have items). Early systems used flat files or rigid hierarchical databases that were hard to query in new ways. The **relational model** (Edgar Codd, 1970) organized data into simple tables with a formal, flexible query language — and it has been the default way to store important data ever since.

11.2 The relational model

Data lives in **tables**. Each **row** is one record; each **column** is one attribute. The table's **schema** defines its columns and their types up front. Two special ideas create relationships:

- **Primary key (PK):** a column (often `id`) that uniquely identifies each row.
- **Foreign key (FK):** a column that references another table's primary key — this is how tables link together.



11.3 SQL — the query language

SQL (Structured Query Language) is how you talk to a relational database. It is **declarative**: you state *what* you want, and the database decides *how* to get it. SQL has three main jobs:

- **Define** the schema (DDL): `CREATE`, `ALTER`, `DROP`.
- **Change** data (DML): `INSERT`, `UPDATE`, `DELETE`.
- **Query** data (DQL): `SELECT`.

```

-- create a table
CREATE TABLE users (
  id    INT PRIMARY KEY,
  name  VARCHAR(100),
  email VARCHAR(255) UNIQUE
);

-- read: all posts by user 42, newest first
SELECT title, body
FROM   posts
WHERE  user_id = 42
ORDER  BY created_at DESC;
  
```

11.4 JOINS — the relational superpower

Because data is split across related tables, you need a way to recombine it. A **JOIN** stitches rows from two tables together on a matching key. This is what makes relational databases so powerful: you store each fact once, then combine tables flexibly at query time.

```

-- every comment, with its post title and the commenter's name
SELECT posts.title, users.name, comments.text
FROM   comments
JOIN   posts ON comments.post_id = posts.id
JOIN   users ON comments.user_id = users.id;
  
```

Relationships come in three shapes: **one-to-one**, **one-to-many** (a user has many posts), and **many-to-many** (students and courses — handled with a third “join table”). JOINS let you ask questions across all of them without duplicating data.

11.5 Schema and constraints: integrity built in

The **schema** fixes the shape of the data, and **constraints** make the database itself enforce correctness:

Constraint	Guarantees
NOT NULL	A value must be present.
UNIQUE	No two rows share this value (e.g. email).
FOREIGN KEY	Referential integrity — you can't reference a user that doesn't exist.
CHECK	A value must satisfy a rule (e.g. <code>price >= 0</code>).

KEY INSIGHT — THE DATABASE GUARDS YOUR DATA

Constraints mean invalid data is rejected at the source, no matter which application or bug tries to write it. That's a major strength — the data stays trustworthy. The flip side is **rigidity**: changing the shape of your data requires a **schema migration**, a deliberate, sometimes tricky operation. Relational databases trade flexibility for integrity.

11.6 ACID — why relational databases are trusted

The deepest reason relational databases are used for money, orders, and other critical data is **ACID transactions** — a set of guarantees that a group of operations behaves correctly:

- **Atomicity**: all-or-nothing — either every step happens or none does.
- **Consistency**: the database moves from one valid state to another (constraints always hold).
- **Isolation**: concurrent transactions don't corrupt each other.
- **Durability**: once committed, data survives crashes.

Classic example: transferring money must debit one account *and* credit another as a single unit — never one without the other. ACID makes that safe. (We devote all of Chapter 14 to transactions and isolation levels.)

11.7 Strengths, weaknesses, and when to use

Strengths	Weaknesses
Structured data with rich relationships; powerful, flexible ad-hoc queries (JOINS); strong consistency and ACID; data integrity via constraints; mature and battle-tested.	Rigid schema (migrations needed); harder to scale horizontally — JOINS and transactions across many machines are painful; can be overkill for simple key-value needs.

KEY INSIGHT — RELATIONAL IS THE SENSIBLE DEFAULT

For most applications with structured, related data that must stay correct — e-commerce, finance, the typical CRUD app — a relational database (PostgreSQL, MySQL) is the right default. Don't reach for something exotic until you have a concrete reason. The one place relational databases struggle is the very thing Part III is about: **scaling the data tier**

horizontally, which is why the coming chapters cover indexing, replication, sharding, and the NoSQL alternatives.

Common mistakes

WATCH OUT

- Duplicating data across rows instead of referencing it with a foreign key.
- Skipping constraints and letting the application be the only guard on data validity.
- Reaching for NoSQL by default when a relational database fits perfectly.
- Treating schema changes as trivial — migrations on large tables need care.
- Assuming a relational database will scale out as easily as the stateless app tier.
- Using a database transaction for unrelated operations, or forgetting one where money/state is involved.

Interview questions

1. Explain the relational model: tables, primary keys, foreign keys.
2. What is a JOIN, and why is it central to relational databases?
3. How do schema and constraints enforce data integrity? Give examples.
4. What does ACID stand for, and why does it matter for something like a bank transfer?
5. What are the main strengths and weaknesses of relational databases?
6. When would you choose a relational database, and when might you not?
7. Why are relational databases harder to scale horizontally?

Hands-on exercises

1. Design tables for a simple blog (users, posts, comments) with primary and foreign keys.
2. Write a SQL query to fetch all comments on a given user's posts.
3. Model a many-to-many relationship (students and courses) with a join table.
4. List the constraints you'd add to an `orders` table to keep it valid.
5. Describe a two-step operation that must be a single ACID transaction, and what breaks without it.

Mini project

MODEL AND QUERY A SMALL DOMAIN

Pick a small domain (a library, a store, a to-do app with shared lists). Design its schema with 3–4 related tables, primary and foreign keys, and sensible constraints. Create the tables in any relational database, insert a handful of rows, and write five queries of increasing difficulty — ending with one that JOINS three tables and uses an aggregate (e.g. count of comments per post). You'll feel how storing data once and combining it at query time is the relational model's core strength.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE DATA MODEL

Design the **relational schema** for your platform: users, posts, follows (a many-to-many self-relationship), likes, and comments, with primary keys, foreign keys, and constraints. Identify which operations need **ACID transactions** (e.g. creating a post and updating a counter together). Note, from your Chapter 5 numbers, which tables will grow largest — these are the ones you'll later index (Chapter 12), replicate (Chapter 16), and shard (Chapter 17).

Chapter summary

- The **relational model** stores data in **tables** of rows and columns, linked by **primary** and **foreign keys** — each fact stored once, then combined at query time.
- **SQL** is a declarative language to define, change, and query data; **JOINS** recombine related tables and are the model's superpower.
- A **schema** and **constraints** make the database enforce data integrity — a strength that brings the rigidity of migrations.
- **ACID** transactions make relational databases trustworthy for critical data like money and orders.
- Relational is the **sensible default** for structured, related, correctness-critical data; its main weakness is **horizontal scaling** — the focus of the rest of Part III.

COMING NEXT

Chapter 12 — Indexing & Query Performance. A correct query can still be painfully slow. We'll see why full table scans kill performance, how indexes turn slow scans into fast lookups, what they cost, and how to read a query plan to find out what your database is really doing.